RECALL ROOM

Flashcard Study Project · Complete source code

A compact web application for practising short question-and-answer cards. The interface is built with HTML, CSS and JavaScript; Flask handles requests and SQLite stores the cards.

Project overview

Main features
• Enter a study topic
• Generate a set of question and answer cards
• Reveal or hide each answer
• Save generated cards in a local SQLite database
• View saved card data through the /history route

Folder structure

```
Recall_Room/\n  app.py\n  requirements.txt\n  templates/\n    index.html\n  static/\n    style.css\n    script.js\n  study_cards.db  (creat
```

How to run the project

```
1. Install Python 3.10 or newer.\n2. Open a terminal in the project folder.\n3. Install dependencies:  pip install -r requirements.txt\n4.
```

Important project note

This version uses a small built-in question bank for the demonstration topics Python, HTML and Java. Other topics receive general sample cards. It does not call an external AI service yet; an AI API can be connected as a future enhancement.

app.py

```
from flask import Flask, render_template, request, jsonify
import sqlite3
from pathlib import Path

app = Flask(__name__)
DATABASE = Path(__file__).with_name("study_cards.db")

# A small starter collection. Add your own subjects here as you study.
STARTER_CARDS = {
    "python": [
        ("What is Python used for?", "Python is used for web development, automation, data analysis and more."),
        ("What is a Python list?", "A list stores items in an ordered collection that can be changed."),
        ("Why do we use functions?", "Functions group reusable instructions under one name.")
    ],
    "html": [
        ("What is the purpose of HTML?", "HTML gives a web page its structure and content."),
        ("Which tag is used for a link?", "The anchor tag, written as &lt;a&gt;, creates a link."),
        ("What is an HTML attribute?", "An attribute gives extra information about an HTML element.")
    ],
    "java": [
        ("What is Java?", "Java is a general-purpose, object-oriented programming language."),
        ("What is an object?", "An object is an instance of a class."),
        ("What is the JVM?", "The Java Virtual Machine runs Java bytecode.")
    ],
    "general": [
        ("What is an algorithm?", "An algorithm is a clear sequence of steps to solve a problem."),
        ("What is a database?", "A database stores information in an organised way."),
        ("What is a variable?", "A variable is a named place used to store a value.")
    ]
}

def create_database():
    with sqlite3.connect(DATABASE) as connection:
        connection.execute("""
            CREATE TABLE IF NOT EXISTS cards (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                topic TEXT NOT NULL,
                question TEXT NOT NULL,
                answer TEXT NOT NULL
            )
        """)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/generate", methods=["POST"])
def generate_cards():
    details = request.get_json(silent=True) or {}
    topic = (details.get("topic") or "").strip()

    if not topic:
        return jsonify({"message": "Please type a subject first."}), 400

    # Use a starter set when the topic is recognised; otherwise show general cards.
    cards = STARTER_CARDS.get(topic.lower(), STARTER_CARDS["general"])

    with sqlite3.connect(DATABASE) as connection:
        connection.executemany(
            "INSERT INTO cards (topic, question, answer) VALUES (?, ?, ?)",
            [(topic, question, answer) for question, answer in cards]
        )

    return jsonify({
        "topic": topic,
        "cards": [{"question": q, "answer": a} for q, a in cards]
    })

@app.route("/history")
def view_history():
    with sqlite3.connect(DATABASE) as connection:
        connection.row_factory = sqlite3.Row
        saved_cards = connection.execute(
            "SELECT topic, question, answer FROM cards ORDER BY id DESC LIMIT 50"
        ).fetchall()

    return jsonify([dict(card) for card in saved_cards])

if __name__ == "__main__":
    create_database()
    app.run(debug=True)
```

templates/index.html

```
&lt;!DOCTYPE html&gt;
&lt;html lang="en"&gt;
&lt;head&gt;
    &lt;meta charset="UTF-8"&gt;
    &lt;meta name="viewport" content="width=device-width, initial-scale=1.0"&gt;
    &lt;title&gt;Recall Room | Study Cards&lt;/title&gt;
    &lt;link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}"&gt;
&lt;/head&gt;
&lt;body&gt;
    &lt;header class="topbar"&gt;
        &lt;span class="brand"&gt;RECALL ROOM&lt;/span&gt;
        &lt;span class="tagline"&gt;A little practice every day.&lt;/span&gt;
    &lt;/header&gt;

    &lt;main class="page"&gt;
        &lt;section class="intro"&gt;
            &lt;p class="eyebrow"&gt;YOUR PERSONAL STUDY SPACE&lt;/p&gt;
            &lt;h1&gt;Learn it. &lt;span&gt;Recall it.&lt;/span&gt;&lt;/h1&gt;
            &lt;p class="description"&gt;
                Choose a subject and make a small set of question cards.
                Try answering before you reveal the explanation.
            &lt;/p&gt;
        &lt;/section&gt;

        &lt;section class="workspace"&gt;
            &lt;form id="cardForm"&gt;
                &lt;label for="topic"&gt;What are you studying today?&lt;/label&gt;
                &lt;div class="form-row"&gt;
                    &lt;input id="topic" name="topic" placeholder="Try Python, HTML or Java" required&gt;
                    &lt;button type="submit"&gt;Make my cards&lt;/button&gt;
                &lt;/div&gt;
            &lt;/form&gt;
            &lt;p id="message" aria-live="polite"&gt;&lt;/p&gt;
            &lt;section id="cardList" class="card-list"&gt;&lt;/section&gt;
        &lt;/section&gt;
        &lt;footer&gt;Recall Room · Keep learning at your own pace&lt;/footer&gt;
    &lt;/main&gt;

    &lt;script src="{{ url_for('static', filename='script.js') }}"&gt;&lt;/script&gt;
&lt;/body&gt;
&lt;/html&gt;
```

static/style.css

```
:root {
    --ink: #24323b;
    --muted: #6d7b82;
    --paper: #fffefa;
    --accent: #276b66;
    --line: #e5e9e4;
}

* { box-sizing: border-box; }

body {
    margin: 0;
    min-height: 100vh;
    color: var(--ink);
    background: #f2f4ef;
    font-family: "Trebuchet MS", Arial, sans-serif;
}

.topbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 22px 7%;
    background: var(--paper);
    border-bottom: 1px solid var(--line);
}

.brand { font-size: 13px; letter-spacing: 2px; font-weight: bold; color: var(--accent); }
.tagline { color: var(--muted); font-size: 13px; }

.page { max-width: 850px; margin: 62px auto 30px; padding: 0 22px; }
.intro { max-width: 620px; margin-bottom: 34px; }
.eyebrow { color: var(--accent); font-size: 11px; letter-spacing: 2px; font-weight: bold; }
h1 { font-size: clamp(38px, 7vw, 58px); line-height: 1.05; margin: 12px 0; }
h1 span { color: var(--accent); }
.description { color: var(--muted); line-height: 1.7; max-width: 540px; }

.workspace {
    padding: 28px;
    background: var(--paper);
    border: 1px solid var(--line);
    border-radius: 16px;
    box-shadow: 0 12px 35px rgba(36, 50, 59, .06);
}

label { display: block; margin-bottom: 12px; font-weight: bold; }
.form-row { display: flex; gap: 10px; }
input {
    flex: 1; min-width: 0; padding: 14px;
    border: 1px solid #d8dfd9; border-radius: 9px;
    font-size: 15px; background: white;
}
button {
    padding: 13px 18px; border: 0; border-radius: 9px;
    color: white; background: var(--accent);
    font-weight: bold; cursor: pointer;
}
button:hover { background: #1d5652; }
#message { min-height: 22px; color: var(--accent); font-size: 14px; }
.card-list { display: grid; gap: 13px; margin-top: 12px; }
.study-card {
    padding: 19px; border: 1px solid var(--line);
    border-radius: 12px; background: #fbfcf8;
}
.study-card h2 { margin: 0 0 12px; font-size: 17px; }
.answer { display: none; color: var(--muted); line-height: 1.6; margin: 12px 0 0; }
.study-card.open .answer { display: block; }
.reveal { padding: 9px 13px; font-size: 13px; }
footer { padding: 28px 0; text-align: center; color: var(--muted); font-size: 12px; }

@media (max-width: 560px) {
    .topbar { padding: 18px 5%; }
    .tagline { display: none; }
    .page { margin-top: 38px; }
    .workspace { padding: 20px; }
    .form-row { flex-direction: column; }
    button { width: 100%; }
}
```

static/script.js

```
const form = document.querySelector("#cardForm");
const message = document.querySelector("#message");
const cardList = document.querySelector("#cardList");

form.addEventListener("submit", async (event) =&gt; {
    event.preventDefault();

    const topic = document.querySelector("#topic").value.trim();
    cardList.replaceChildren();
    message.textContent = "Putting your study cards together...";

    try {
        const reply = await fetch("/generate", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ topic })
        });

        const result = await reply.json();
        if (!reply.ok) {
            throw new Error(result.message || "Something went wrong.");
        }

        message.textContent = `${result.cards.length} cards ready for ${result.topic}.`;

        result.cards.forEach((item, position) =&gt; {
            const card = document.createElement("article");
            card.className = "study-card";

            const question = document.createElement("h2");
            question.textContent = `${String(position + 1).padStart(2, "0")}  ${item.question}`;

            const answer = document.createElement("p");
            answer.className = "answer";
            answer.textContent = item.answer;

            const revealButton = document.createElement("button");
            revealButton.type = "button";
            revealButton.className = "reveal";
            revealButton.textContent = "Reveal answer";

            revealButton.addEventListener("click", () =&gt; {
                card.classList.toggle("open");
                revealButton.textContent = card.classList.contains("open")
                    ? "Hide answer" : "Reveal answer";
            });

            card.append(question, revealButton, answer);
            cardList.append(card);
        });
    } catch (error) {
        message.textContent = error.message;
    }
});
```

requirements.txt

```
Flask&amp;gt;=3.0,&amp;lt;4.0
```